



Universidad Nacional de Costa Rica

Sede Regional Brunca
Campus Coto y Pérez Zeledón

EIF207 Estructuras de Datos

PROYECTO I (15 %)

1. Descripción general

Va a implementar una versión simplificada de *Tetris* en C++, usando exclusivamente las estructuras de datos lineales estudiadas en las semanas 1 a 8 del curso (tipos abstractos de datos, listas, pilas, colas y algoritmos de ordenamiento). El objetivo no es reproducir el Tetris comercial con todas sus reglas oficiales, sino resolver cada mecánica del juego con la estructura de datos que le corresponde de forma natural, y poder explicar y defender esa decisión.

El juego consiste en un tablero de 10 columnas por 20 filas donde caen piezas de 4 bloques (tetrominós) que el jugador debe acomodar para completar filas horizontales. Cada fila completa se elimina y las filas superiores descienden. El juego termina cuando una pieza nueva no tiene espacio para aparecer.

2. Objetivos de aprendizaje

- Implementar y utilizar correctamente una **cola** propia (FIFO) para gestionar la secuencia de piezas futuras.
- Implementar y utilizar correctamente una **pila** propia (LIFO) para la pieza en espera (*hold*).
- Implementar una **lista doblemente enlazada** propia para navegar hacia atrás y hacia adelante en el historial de movimientos, y reproducir la partida completa al finalizar.
- Implementar una segunda estructura de **cola ordenada** por tiempo/condición para gestionar eventos programados del juego.
- Implementar una **lista enlazada** para representar el tablero como una secuencia de filas, insertando y eliminando nodos al limpiar líneas.
- Implementar al menos **dos algoritmos de ordenamiento** distintos y compararlos empíricamente para ordenar la tabla de mejores puntajes.
- Analizar la **complejidad asintótica** de las operaciones implementadas y contrastarla contra mediciones reales de tiempo de ejecución.

3. Especificaciones funcionales obligatorias

3.1 Piezas y generación (cola)

Existen las 7 piezas clásicas (I, O, T, S, Z, J, L). La secuencia de piezas futuras se genera por “bolsas”: cada bolsa contiene las 7 piezas en orden aleatorio, y una nueva bolsa se genera y se

agrega a la cola cuando la anterior se agota. La cola debe ser una implementación propia (nodo + puntero, o arreglo circular propio); **no se permite** `std::queue` ni `std::deque` para esta cola. El jugador debe poder ver, en pantalla, al menos las siguientes 3 piezas de la cola.

3.2 Pieza en espera y repetición de partida (pila + lista doblemente enlazada)

El jugador puede enviar la pieza actual a una “*casilla de espera*” (*hold*) para usarla más adelante; esa casilla se implementa como una **pila propia de capacidad 1**, no `std::stack`.

Además, cada movimiento relevante (mover, rotar, bajar, colocar) se debe registrar como un nodo en una **lista doblemente enlazada propia** (no `std::list`). Mientras la partida está en curso, el jugador debe poder **retroceder** (**deshacer**) y **avanzar** (**rehacer**) cualquier cantidad de pasos usando esa lista, no solo el último movimiento. Al finalizar la partida (por *game over*) debe poder reproducir la partida completa recorriendo la lista desde el primer nodo, mostrando el tablero paso a paso.

3.3 Cola de eventos programados

El juego debe programar al menos **3 tipos de eventos futuros** (por ejemplo: aumento de la velocidad de caída cada cierto número de líneas o segundos, aparición de una pieza especial que limpia una fila al colocarse, u otro efecto que usted defina y documente) en una **cola de eventos propia**, ordenada siempre por el momento en que cada evento debe dispararse. Cada evento nuevo se inserta en la posición correcta según su momento de disparo (no se permite insertar al final y reordenar todo cada vez); cuando se cumple la condición del evento que está al frente, se extrae de la cola y se aplica su efecto. **No se permite** usar `std::priority_queue`.

3.4 Tablero (lista enlazada)

El tablero se representa internamente como una **lista enlazada de 20 nodos**, donde cada nodo es una fila de 10 celdas. Cuando una o más filas quedan completas, esos nodos se eliminan de la lista y se insertan al inicio la misma cantidad de nodos de fila vacía, de modo que el resto de filas “caiga” de forma consistente con el modelo de lista. **No se permite** representar el tablero únicamente como un arreglo bidimensional o un `std::vector<std::vector<...>>` como estructura principal: la lista debe ser la representación real, aunque internamente cada fila use un arreglo fijo de 10 celdas.

3.5 Caída, colisión y rotación

Cada pieza cae una fila cada cierto intervalo de tiempo fijo (salvo cuando la cola de eventos programados lo modifique). El jugador puede moverla horizontalmente, rotarla y forzar su descenso. La rotación se implementa con las **4 orientaciones precalculadas** de cada pieza (sin sistema de “*wall kick*”): si no hay espacio para la orientación siguiente, la pieza simplemente no rota. El juego termina cuando una pieza nueva no puede colocarse en su posición inicial.

3.6 Puntaje y tabla de mejores puntajes (ordenamiento)

Cada línea limpiada suma puntos (puede definir su propia fórmula, indicándola en el informe). Al finalizar una partida, si el puntaje califica entre los **10 mejores**, se guarda en un archivo junto con el nombre del jugador. Cada vez que se muestra la tabla de mejores puntajes, debe estar ordenada de mayor a menor. Debe implementar **al menos dos algoritmos de ordenamiento distintos** (por ejemplo, uno de complejidad $O(n^2)$ como inserción o burbuja, y uno de complejidad $O(n \log n)$ como *quicksort* o *mergesort*), permitir seleccionar cuál se usa, y **no puede usar** `std::sort` para esta parte.

4. Restricciones técnicas

Lenguaje: C++. **Entorno / compilador:** Zinjal.

El proyecto es individual. Cada estudiante desarrolla, entrega y defiende su propia solución.

No se permite usar `std::stack`, `std::queue`, `std::deque`, `std::list`, `std::priority_queue` ni `std::sort` para las estructuras y el ordenamiento que este enunciado pide implementar. **Sí puede usar** `std::string`, `std::vector` para colecciones auxiliares que no sean las estructuras evaluadas (por ejemplo, para leer un archivo), y la biblioteca estándar para entrada/salida y manejo de tiempo.

El estudiante debe estar en capacidad de **explicar y defender cualquier parte del código** durante la defensa oral. Cualquier indicio de código compartido entre estudiantes distintos, o de código no propio sin atribución, se tramita según el *reglamento de fraude académico* de la Universidad Nacional.

4.1 Control de autoría y trabajo incremental (control de versiones)

Debe desarrollar el proyecto usando **control de versiones con git** desde el inicio del desarrollo, con commits incrementales que reflejen el avance real; no se acepta un repositorio con un solo commit final ni un historial reescrito de una sola vez. Puede usar un repositorio local o en una plataforma como GitHub o GitLab (público, o privado agregando al profesor como colaborador). Como parte de la entrega debe incluir el **historial de commits** (por ejemplo, la carpeta `.git` completa dentro del `.zip`, o la salida de `git log -stat -date=short` en un archivo de texto). Este historial se revisará como respaldo adicional de que el trabajo se desarrolló durante el periodo asignado, y podrá consultarse antes o durante la defensa oral.

5. Interfaz y elementos gráficos (obligatorio)

La **interfaz gráfica es obligatoria**: debe usar **SFML** para dibujar el tablero, las piezas, la cola de próximas piezas, la pieza en *hold* y el puntaje, en una ventana real (no se acepta consola/ANSI como entrega final; puede usarla como prototipo intermedio durante el desarrollo). Debe incluir **animación de caída y de limpieza de línea**, pantallas de inicio, pausa y fin de juego, y controles visibles para reproducir, retroceder y avanzar en el *replay* al finalizar la partida.

6. Entregables

1. **Código fuente completo**, organizado en archivos `.h/.cpp` por módulo (por ejemplo: cola de piezas, pilas, replay/lista doble, cola de eventos, tablero/lista, ordenamiento, interfaz gráfica), con un **README** que indique cómo compilar y ejecutar el programa y qué librería gráfica se usó.
2. **Historial de control de versiones** (carpeta `.git` o exportación del `git log`), según lo indicado en la sección 4.1.
3. **Archivo de mejores puntajes** generado por el propio programa (puede incluir uno de ejemplo).
4. **Informe corto** en PDF o Word (máximo 6 páginas) con: diagrama o descripción de cómo se relacionan las estructuras entre sí, complejidad teórica (notación asintótica) de cada operación implementada, tabla o gráfico de tiempos medidos al ordenar la tabla de puntajes con distintos tamaños de datos (por ejemplo 10, 100, 1000, 10 000 registros de prueba), y las respuestas a las preguntas de análisis de la sección 7.
5. Un **archivo comprimido** (`.zip`) que agrupe todo lo anterior.

7. Preguntas de análisis obligatorias (para el informe)

1. Compare el tiempo de ordenar la tabla de puntajes con el algoritmo de complejidad $O(n^2)$ contra el de $O(n \log n)$, para tamaños crecientes de datos de prueba. ¿A partir de qué tamaño se nota la diferencia? ¿Coincide con lo que predice la notación asintótica?
2. Explique por qué representar el tablero como una lista enlazada de filas es una forma razonable de modelar la limpieza de líneas, y qué costo (en operaciones) tiene en su implementación insertar una fila vacía y eliminar una fila completa.
3. ¿Por qué el *replay* requiere una lista doblemente enlazada y no una simplemente enlazada? ¿Qué costo tiene, en su implementación, retroceder o avanzar un paso?
4. ¿Cómo garantiza su implementación que la cola de eventos programados siempre mantenga al frente el evento más próximo a dispararse? ¿Qué complejidad tiene insertar un nuevo evento?

8. Tabla de calificación

El porcentaje final de este rubro corresponde al **15 % de la nota del curso**.

Importante: si el proyecto **no cumple las restricciones técnicas** (sección 4) — por ejemplo, usar estructuras prohibidas de la STL, no usar ZinjaI/C++ o entregar en consola en lugar de la interfaz gráfica obligatoria — **no se revisa** y se califica con cero.

La calificación se compone de un **puntaje del proyecto** (0–100 puntos, apartados A, B y C) que luego se **multiplica por el factor de defensa** (apartado D) para obtener la nota final de cada estudiante:

Nota final del proyecto = (Puntaje del proyecto, 0–100) $\times$ (Factor de defensa, 0,00–1,00).

Rubros del puntaje del proyecto (0–100)

Rubro	Valor
A. Documentación (informe, README, comentarios y evidencia de git)	
Informe de análisis (complejidad teórica vs. tiempos medidos, respuestas de la sección 7)	7
Calidad de código y trabajo incremental (organización, sin fugas de memoria, historial de commits real)	10
B. Planteamiento y solución adaptada (la estructura correcta para cada mecánica)	
Cola de piezas propia (bolsa de 7 con mezcla aleatoria, sin <code>std::queue</code>)	8
Pila propia de <i>hold</i> (pieza en espera)	4
Replay: lista doblemente enlazada propia con deshacer/rehacer (varios pasos) y reproducción completa	16
Cola de eventos programados (nivel/power-ups), siempre ordenada por momento de disparo	9
Tablero como lista enlazada de filas: inserción de fila vacía y eliminación de fila(s) completa(s)	12
C. Funcionamiento del aplicativo	
Mecánica de juego: caída, colisión, rotación fija (4 orientaciones), condición de fin de juego	11
Persistencia del marcador + al menos 2 algoritmos de ordenamiento propios + comparación empírica	12
Interfaz gráfica obligatoria (SFML): tablero, piezas, siguientes, <i>hold</i> , puntaje, animaciones y controles de replay	11
SUBTOTAL — Puntaje del proyecto	100

El puntaje del proyecto se obtiene de la suma de los rubros A, B y C. Ese subtotal (0–100) se multiplica luego por el factor de defensa del apartado D para obtener la nota final individual de cada estudiante.

D. Defensa del proyecto (factor multiplicador 0,00–1,00)

La defensa **no suma puntos**: **multiplica** el puntaje obtenido en los rubros A, B y C. Se evalúa de forma individual mediante una sesión de preguntas del cuerpo docente sobre cualquier componente del entregable. El factor se obtiene sumando los cuatro criterios siguientes y dividiendo el total entre 100.

Criterio	Valor	Puntos
D1. Dominio técnico: explica qué hace cada componente, cómo se configuró y por qué funciona.	30	
D2. Justificación de decisiones: argumenta las alternativas descartadas, el modelo de amenazas y los controles aplicados.	25	
D3. Respuesta a preguntas imprevistas: modifica, depura o razona en vivo sobre su propia solución.	25	
D4. Comunicación: claridad al exponer, uso correcto de la terminología y evidencia de trabajo propio en el desarrollo.	20	
TOTAL	100	

Factor de defensa = Total obtenido ÷ 100.

Factor	Descriptor
0,00	No se presenta a la defensa, o no logra explicar ningún componente del entregable.
0,01–0,40	Reconoce el proyecto pero no puede explicar cómo funciona; respuestas memorizadas o incorrectas.
0,41–0,60	Explica el funcionamiento general, pero no justifica decisiones ni responde ante variaciones del escenario.
0,61–0,80	Domina su parte del proyecto y justifica la mayoría de las decisiones; presenta dudas puntuales.
0,81–0,95	Domina el proyecto completo, justifica decisiones técnicas y responde con solvencia a preguntas imprevistas.
0,96–1,00	Dominio total: razona sobre el proyecto en vivo, propone mejoras y discute limitaciones y riesgos residuales.

Condiciones de la defensa

1. La defensa es **obligatoria**. La ausencia injustificada implica factor **0,00** para esa persona, independientemente del puntaje del entregable.
2. Si el estudiante no logra explicar el funcionamiento de un componente que aparece en su entrega (código, script, configuración o documento), el factor **no podrá superar 0,50**, sin importar la calidad del entregable.
3. El uso de herramientas de asistencia (incluida la inteligencia artificial) es permitido durante el desarrollo, pero **no durante la defensa**. Lo que se evalúa es la comprensión del estudiante, no la autoría de la herramienta.
4. El cuerpo docente puede solicitar **modificaciones en vivo** sobre el entregable como parte del criterio D3.
5. El factor de defensa aplica **únicamente** sobre la nota del proyecto final; no afecta otros rubros de evaluación del curso.

Ejemplo: un entregable calificado con 100 puntos, defendido por un estudiante con un factor de 0,70, obtiene una nota final de **70**.

9. Entrega, defensa y logística

La entrega se realiza preferiblemente por el **aula virtual**; si el profesor solicita envío por correo, el asunto y el nombre del archivo comprimido deberán seguir el formato indicado por el profesor siguiendo como referencia el esquema **Proyecto1 [Nombre] [Cédula]** de la carta al estudiante.

El mismo día de la entrega se realiza una **defensa oral**, donde deberá mostrar el programa funcionando, explicar sus decisiones de diseño y responder preguntas sobre el código propio; el profesor podrá revisar el historial de commits antes o durante la defensa. **La presentación de este proyecto es requisito indispensable para aprobar el curso.**